DIGIZAFE — TEAM MEMBER 2 MASTER TECHNICAL NOTES

Your role

Automation Engineer

Your official responsibility:

> **"I built the Playwright remediation engine that navigates data broker websites and automates privacy opt-out requests."**

This means your professor should be able to ask you about:

1. **Why browser automation is necessary**
2. **How Playwright works**
3. **How the remediation pipeline works**
4. **How data broker forms are discovered and filled**
5. **CAPTCHA/manual intervention**
6. **DOM interaction**
7. **Success/failure detection**
8. **Celery integration**
9. **Browser isolation**
10. **Security and privacy of automated remediation**
11. **Timeouts and resource management**
12. **Limitations of heuristic browser automation**

Your central story is:

> **Discover → identify exposure → create remediation job → launch isolated browser → navigate broker → locate form → fill PII → handle CAPTCHA/manual step → submit → verify result → persist state → re-score**

PART I — UNDERSTAND YOUR ROLE IN THE WHOLE SYSTEM

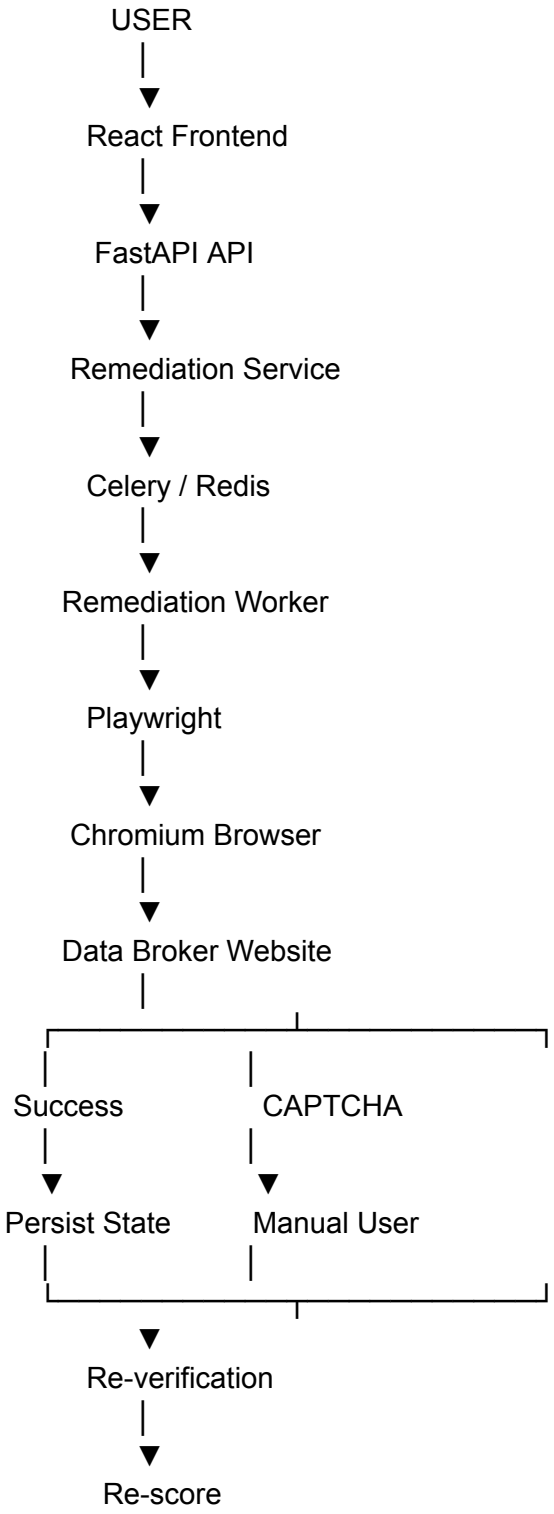
You are responsible for the **action layer**.

Team Member 3 finds data.

Team Member 1 determines identity/risk.

You actually attempt to remove the exposed data.

The overall architecture is:



The project describes this as the core innovation because DigiZafe does not merely discover exposure; it attempts to perform the actual privacy opt-out workflow.

PART II — WHAT PROBLEM ARE YOU SOLVING?

The traditional problem

Suppose a user discovers that their information appears on:

DataBrokerA
DataBrokerB
DataBrokerC
DataBrokerD

Normally they have to:

Open website
↓
Find opt-out page
↓
Search for themselves
↓
Enter name
↓
Enter email
↓
Enter location
↓
Complete CAPTCHA
↓
Submit
↓
Wait for confirmation
↓
Repeat

For ten or twenty brokers, this becomes extremely tedious.

DigiZafe attempts to automate this workflow.

The project's stated motivation is that finding exposure can take hours while manually removing it can require extensive form filling.

PART III — WHAT IS AUTOMATED REMEDIATION?

Remediation

In cybersecurity/privacy terminology, remediation means:

> **Taking an action to reduce or eliminate an identified risk.**

In DigiZafe:

Finding
↓
Privacy risk
↓
Remediation recommendation
↓
Opt-out request
↓
Data removal

So:

Discovery

> "Your information is here."

Risk scoring

> "This exposure is important."

Remediation

> **"Let's attempt to remove it."**

PART IV — WHY PLAYWRIGHT?

This is probably your **#1 viva question**.

What is Playwright?

Playwright is a browser automation framework that allows software to control real browser engines programmatically.

DigiZafe uses it to automate Chromium.

Conceptually:

Python
↓
Playwright
↓
Chromium
↓
Website

Instead of making only HTTP requests:

requests.get(url)

the system can actually:

Open browser
↓
Load page
↓
Execute JavaScript
↓
Render DOM
↓
Click buttons
↓
Fill forms
↓
Submit forms

Why not use `requests`?

This is an important distinction.

Modern data broker sites may contain:

- JavaScript
- dynamically generated forms
- client-side validation
- hidden fields
- dynamically loaded content
- CAPTCHA components
- JavaScript event handlers

A simple HTTP client may retrieve:

```
<html>
   ...
</html>
```

but not reproduce the full browser interaction required by the site.

Playwright provides a real browser environment.

The project explicitly describes the remediation engine as using Playwright to render real DOMs and interact with JavaScript-heavy forms.

PART V — YOUR CORE FILE

The most important file you need to know is:

backend/app/remediation/runners/playwright_runner.py

The Sprint 7 specification identifies this as the Playwright Green-broker runner.

You should be able to explain almost every major operation in this file.

PART VI — THE REMEDIATION ARCHITECTURE

Your subsystem has several layers.

backend/app/

```
│
├── api/v1/
│   └── remediation.py
│
├── services/
│   └── remediation_service.py
│
├── repositories/
│   └── remediation_repository.py
│
├── models/
│   └── remediation.py
│
├── schemas/
│   └── remediation.py
│
├── remediation/
│   ├── broker_registry.py
│   ├── runners/
│   │   └── playwright_runner.py
│   ├── generators/
│   └── playbooks/
│
└── tasks/
    └── remediation_tasks.py
```

The Sprint 7 file checklist explicitly identifies these components.

PART VII — WHAT EACH COMPONENT DOES

1. `remediation.py`

This is the API layer.

It exposes endpoints for:

- starting opt-out jobs
- checking state
- CAPTCHA handling
- manual completion
- freeze checklist

- rights requests
- complaints
- verification

For example:

POST /api/v1/remediation/jobs/broker-optout

starts a broker opt-out job.

---

2. `remediation_service.py`

This is the business-logic layer.

It coordinates:

API
↓
RemediationService
↓
Database
↓
Celery

The API should not contain all remediation logic itself.

---

3. `remediation_repository.py`

Handles database access.

It persists things such as:

- jobs
- job items
- broker states
- CAPTCHA/manual states
- generated requests

---

4. `remediation.py` model

Stores persistent remediation information in PostgreSQL.

---

5. `broker_registry.py`

This is extremely important.

It defines which brokers are allowed to be automated.

The registry is loaded from:

shared/config/broker_registry/brokers_green.json

The registry loader filters for brokers whose legality is marked `"green"` and optionally enabled.

---

6. `playwright_runner.py`

This is **your core automation engine**.

It actually controls Chromium.

---

7. `remediation_tasks.py`

This connects the remediation workflow to Celery.

Conceptually:

FastAPI
↓
RemediationService
↓
Celery task
↓

Worker
↓
Playwright

PART VIII — WHAT IS A "GREEN BROKER"?

This is an important project-specific concept.

The system does **not** blindly automate every website.

The Sprint 7 design specifically limits automated remediation to **Green brokers**.

The registry contains broker metadata.

Conceptually:

```
{
  "id": "example_broker",
  "enabled": true,
  "legality": "green"
}
```

The runner only operates on approved broker entries.

WHY?

Because automated interaction with arbitrary websites introduces:

- legal risk
- security risk
- unexpected behavior
- SSRF concerns
- unreliable forms
- potentially harmful automation

Therefore:

> **The automation boundary is intentionally restricted.**

PART IX — COMPLETE PLAYWRIGHT WORKFLOW

Memorize this.

1. Receive remediation job
   ↓
2. Validate broker
   ↓
3. Confirm verified identifier
   ↓
4. Launch isolated Chromium
   ↓
5. Navigate to broker opt-out URL
   ↓
6. Wait for page load
   ↓
7. Inspect DOM
   ↓
8. Detect CAPTCHA
   ↓
9. Identify required form fields
   ↓
10. Map logical fields → DOM selectors
   ↓
11. Fill PII
   ↓
12. Submit
   ↓
13. Inspect response DOM
   ↓
14. Search success hints
   ↓
15. Mark success/failure/manual
   ↓
16. Persist state
   ↓
17. Re-score / re-verify

The documented runner specifically performs DOM scanning, field mapping, form submission and post-submit success detection.

PART X — STEP 1: VALIDATE THE BROKER

Before opening the browser:

broker_id
  ↓
broker registry
  ↓
Is it enabled?
  ↓
Is it Green?
  ↓
YES → continue
NO → reject

This prevents the worker from blindly visiting arbitrary destinations.

PART XI — STEP 2: VERIFIED IDENTIFIER

The remediation workflow requires a verified identifier.

The Sprint 7 safety rules explicitly state:

G1 verified identifier only

along with:

Green brokers only
Playwright only in worker
Consent + egress ledger
CapSolver never required

This is important.

You shouldn't say:

> "The user can enter any random email and we scrape it."

Instead:

> **"Automated remediation is tied to a verified identifier and approved broker workflow."**

PART XII — STEP 3: LAUNCH CHROMIUM

The worker launches Playwright Chromium.

Conceptually:

browser = await playwright.chromium.launch(...)

Then:

Browser
↓
Context
↓
Page

PART XIII — BROWSER CONTEXT

A browser context is similar to an isolated browser session.

It can contain:

- cookies
- local storage
- session state
- cache
- permissions

WHY CAN'T YOU REUSE CONTEXTS BETWEEN USERS?

This is a major security question.

Imagine:

User A
↓
Browser context
↓
cookies
session
local storage

Then accidentally:

User B
↓
same context

User B could inherit User A's:

- cookies
- session
- authentication state
- cached information

That could create a cross-user privacy leak.

Therefore:

> **Each remediation execution must use an isolated browser context.**

The project documentation explicitly identifies cross-user context reuse as forbidden for this reason.

PART XIV — HEADLESS MODE

The configuration includes:

PLAYWRIGHT_HEADLESS=true

The browser therefore runs without displaying a GUI in the worker environment.

Why headless?

Because this is a server-side worker.

You don't want:

Docker container
↓
physical browser window

You want:

Worker
↓
headless Chromium
↓
website

PART XV — CUSTOM USER AGENT

The runner uses a custom User-Agent as part of its browser configuration.

A User-Agent tells a website what kind of client is making the request.

For example:

Browser
Operating system
Browser version

However, be careful in viva:

Don't say:

> "This bypasses bot detection."

Say:

> **"The runner configures a browser-like User-Agent, but this is not a guarantee of bypassing anti-bot systems."**

PART XVI — DOM

This is essential.

What is DOM?

DOM = **Document Object Model**

When a browser loads:

<input id="email">
<button id="submit">

the browser converts the page into a structured object tree.

Playwright can interact with that structure.

Conceptually:

HTML
↓
Browser
↓
DOM
↓
Playwright Locator
↓
Interaction

PART XVII — WHY DOM AUTOMATION IS HARD

Every broker may use different HTML.

Broker A:

<input id="email">

Broker B:

<input name="user_email">

Broker C:

<input class="email-field">

Same logical field:

EMAIL

Different implementation.

Therefore DigiZafe separates:

Logical field

email

from:

DOM selector

#email
[name="user_email"]
.email-field

The project calls this a **logical-to-selector mapping**.

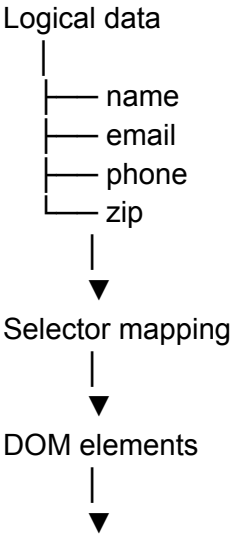
PART XVIII — FORM FILLING

The runner can fill PII fields such as:

Name
Email
Phone
ZIP

The documentation states that the runner uses `.select_option()` for `<select>` elements and `.fill()` for normal fields.

Conceptually:



PART XIX — `.fill()` VS `.select_option()`

Professor:

> "Why can't you use `.fill()` everywhere?"

Because a `<select>` element isn't a normal text input.

For example:

```
<select name="state">
   <option value="CA">California</option>
   <option value="NY">New York</option>
</select>
```

You should select:

CA

rather than trying to type text into the select control.

Hence:

<input> → fill()
<select> → select_option()

PART XX — PAGE LOAD

The runner explicitly waits for:

domcontentloaded

before continuing.

Why?

Because you don't want to query the page before the initial DOM is available.

Conceptually:

navigate()
  ↓
wait for DOMContentLoaded
  ↓
inspect DOM

The viva guide identifies this as part of preventing DOM race conditions.

PART XXI — WHAT IS A DOM RACE CONDITION?

Imagine:

Page starts loading
      ↓
Your code immediately searches for email field
      ↓
Field doesn't exist yet
      ↓
ERROR

That's a race between:

Page rendering

and:

Automation code

Playwright's waiting/locator behavior reduces this problem.

The runner additionally has explicit timeouts.

PART XXII — TIMEOUTS

This is very important.

Imagine:

Broker website never responds

If the worker waits forever:

Worker
↓
Browser
↓
waiting...
↓
waiting...
↓
waiting...

eventually the worker can become unusable.

Therefore the runner uses hard timeouts.

The Sprint 7 design specifies:

BROKER_RUNNER_TIMEOUT_SECONDS=90

and the runner rules state:

Timeouts are hard-enforced.

PART XXIII — CAPTCHA DETECTION

This is one of your biggest viva topics.

Modern websites can contain:

- reCAPTCHA
- hCaptcha
- Cloudflare Turnstile

The runner scans page content for indicators such as:

recaptcha
hcaptcha
cf-turnstile

Conceptually:

Page
↓
DOM / HTML
↓
Search CAPTCHA indicators
↓
CAPTCHA detected?
├── NO → continue
└── YES → captcha_needed

PART XXIV — IMPORTANT:

DO NOT SAY "WE BYPASS CAPTCHA"

This is a very important presentation correction.

Some older project descriptions loosely describe CAPTCHA token injection.

But the stronger/current Sprint 7 safety model says:

CAPTCHA
↓
captcha_needed
↓
user/manual action
↓
resume

and:

CapSolver optional
default = manual/open_in_browser

The defense deck also explicitly says the project should **avoid unethical CAPTCHA bypassing**.

So your safest answer is:

> **"The system detects CAPTCHA and supports a manual/user-assisted path. The current default does not require an automated CAPTCHA-solving service."**

PART XXV — CAPTCHA WORKFLOW

Memorize this exactly:

Job
↓
Broker page
↓
CAPTCHA detected
↓
Job item = captcha_needed
↓
GET /remediation/captcha
↓
User receives instructions/page URL
↓
User completes CAPTCHA
↓
POST /remediation/captcha/{id}
↓
manual_done / solve

↓
Worker resumes

This flow is explicitly specified in Sprint 7.

PART XXVI — WHY MANUAL CAPTCHA?

Because CAPTCHA is designed specifically to distinguish automated clients from humans.

Trying to universally defeat it would:

- be unreliable
- be ethically problematic
- violate site expectations
- make the automation brittle

Therefore the architecture treats CAPTCHA as a **human-in-the-loop boundary**.

That's a much better engineering design.

PART XXVII — CAPSOLVER

There is a configuration option:

FEATURE_CAPSOLVER=false

and:

CAPTCHA_MODE=manual

The documented options include:

manual
open_in_browser
capsolver

but CapSolver is not required by the architecture.

So if asked:

> "Do you automatically solve every CAPTCHA?"

Answer:

> **"No. CAPTCHA is treated as an intervention point. The default path is manual/user-assisted; automated solving is optional and feature-gated rather than being a mandatory dependency."**

PART XXVIII — SUCCESS DETECTION

This is a very clever part of the implementation.

After submitting the form, how does the program know that the removal request succeeded?

It cannot simply assume:

HTTP 200 = success

because:

HTTP 200

could simply mean:

> "The page loaded successfully."

It doesn't necessarily mean:

> "Your information has been removed."

SUCCESS HINTS

The runner parses the resulting DOM/body and searches for known strings such as:

Your request has been received

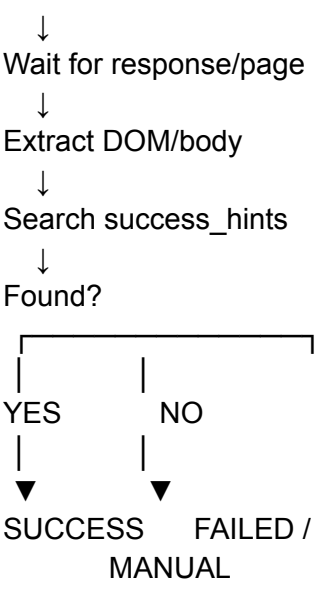
or:

Your information has been removed

The project documentation calls these:

success_hints

PART XXIX — SUCCESS DETECTION FLOW



The viva material specifically identifies post-submit HTML parsing against known success hints as the mechanism.

PART XXX — WHY IS THIS A HEURISTIC?

Because websites don't provide one universal API like:

```
{
  "removed": true
}
```

Instead, each website may say:

"Your request has been submitted."

or:

"We'll process your request."

or:

"Removal successful."

Therefore the system uses predefined textual signals.

This makes the process:

heuristic rather than mathematically deterministic.

The project explicitly describes the form submission as heuristic.

PART XXXI — WHAT IF THE WEBSITE CHANGES?

This is one of your biggest limitations.

Suppose the website changes:

<input id="email">

to:

<input id="contact_email">

Your old selector:

#email

fails.

Therefore:

Broker changes HTML

↓
Locator fails
↓
Automation cannot continue
↓
MANUAL_NEEDED

The project explicitly acknowledges that heuristic DOM scraping is brittle and falls back to manual intervention when the site's structure changes.

PART XXXII — SHADOW DOM

Another advanced question.

A website may render content inside a:

Shadow DOM

Basic selectors may not behave as expected depending on how the page is structured.

The project specifically identifies shadow-DOM and cross-origin iframe cases as edge cases that can cause the runner to fall back to manual intervention.

PART XXXIII — IFRAMES

A form could be inside:

<iframe>

The page you're controlling may not directly contain the form elements in its main DOM.

Therefore the automation needs iframe-aware handling if supported.

Cross-origin restrictions make this more complicated.

For the current design:

> If basic locators can't safely interact with the required structure, the job should degrade to manual handling.

PART XXXIV — DRY RUN

This is an extremely useful feature.

The runner supports:

dry_run=true

The documented rule is:

> **dry_run fills but does not submit.**

So:

Dry run
↓
Open website
↓
Locate form
↓
Fill fields
↓
DO NOT SUBMIT

WHY IS DRY RUN IMPORTANT?

Testing automated privacy requests against real websites can be risky.

You don't want:

Developer testing
↓
actual deletion request

every time.

Dry-run allows:

Test selectors
Test form mapping

Test browser workflow

without executing the final action.

PART XXXV — SAFETY BOUNDARIES

The remediation engine has several important safety controls.

1. Verified identifier

Only verified identity information should be used.

2. Green brokers only

No arbitrary websites.

3. Worker-only browser execution

Playwright runs in the background worker rather than directly inside the API request.

4. Consent + egress ledger

The broker run is auditable.

5. CAPTCHA manual path

CAPTCHA does not force the system into unsafe automation.

These are explicitly documented safety requirements.

PART XXXVI — WHY PLAYWRIGHT RUNS IN CELERY

This is an important architecture question.

Why not:

FastAPI
↓
Playwright

?

Because browser automation is:

- slow
- memory-intensive
- unpredictable
- long-running

If FastAPI directly launches Chromium:

HTTP request
↓
Chromium
↓
60 seconds
↓
response

the API layer becomes burdened by browser resources.

Instead:

FastAPI
↓
create job
↓
Redis
↓
Celery
↓
Playwright

The API returns quickly.

PART XXXVII — WHY A SEPARATE REMEDIATION WORKER?

The Sprint 7 deployment plan supports an optional dedicated:

remediation-worker

service with:

--concurrency=1

and Playwright installed.

This is important because Chromium is memory-heavy.

PART XXXVIII — WHY CONCURRENCY = 1?

Suppose one Chromium instance consumes substantial memory.

If:

worker concurrency = 10

you could have:

Browser 1
Browser 2
Browser 3
...
Browser 10

inside one container.

This could cause:

RAM exhaustion
↓
OOM
↓
worker crash

The project therefore limits the remediation worker to one concurrent task in the documented deployment configuration.

The viva guide explicitly identifies this as the strategy for controlling Playwright memory usage.

PART XXXIX — RESOURCE MANAGEMENT

Your browser lifecycle should conceptually be:

Start Playwright
↓
Launch Browser
↓
Create Context
↓
Create Page
↓
Execute Job
↓
Close Page
↓
Close Context
↓
Close Browser

Why?

To avoid:

- memory leaks
- stale sessions
- open browser processes
- cross-user state

PART XL — USER DATA DIRECTORY

The configuration includes:

PLAYWRIGHT_USER_DATA_DIR=/tmp/digizafe-pw

with the explicit rule:

> **Never share browser context across users.**

This is part of maintaining session isolation.

PART XLI — BROKER REGISTRY

You should know this file:

shared/config/broker_registry/brokers_green.json

It acts like a catalog.

Conceptually:

```
Broker Registry
│
├── broker ID
├── URL
├── enabled
├── legality
├── opt-out configuration
└── strategy
```

The loader returns only enabled Green brokers for normal automation.

PART XLII — WHY A REGISTRY?

Without a registry, you might hardcode:

```
if broker == "A":
    ...
elif broker == "B":
    ...
elif broker == "C":
    ...
```

This becomes difficult to maintain.

Instead:

Broker Registry
↓
Broker Metadata
↓
Runner

This provides separation between:

Broker configuration

and

Automation engine.

PART XLIII — GENERIC VS BROKER-SPECIFIC AUTOMATION

There are two levels.

Generic logic

Things such as:

launch browser
navigate
detect CAPTCHA
fill field
submit
check success

are common.

Broker-specific logic

Things such as:

email selector
phone selector
name selector
submit selector
success text

may vary.

Therefore the architecture uses broker registry/configuration rather than rewriting the entire runner for every website.

PART XLIV — REMEDIATION STATE MACHINE

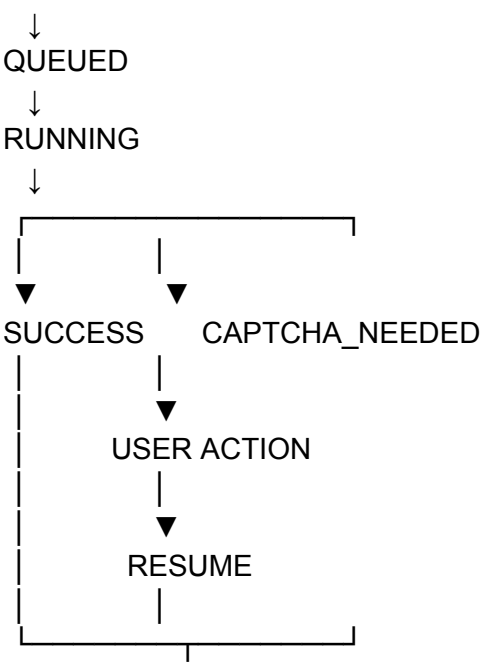
This is extremely important.

A job isn't simply:

SUCCESS / FAILURE

It can have intermediate states.

Conceptually:



▼
VERIFY
│
▼
COMPLETED

Or:

RUNNING
↓
MANUAL_NEEDED

if the automated workflow cannot continue.

PART XLV — CAPTCHA STATE

The documented workflow specifically uses:

captcha_needed

as a job-item state.

This is better than treating CAPTCHA as a generic exception.

Why?

Because the system knows:

> "The job isn't necessarily failed. It needs human intervention."

That distinction is important.

PART XLVI — MANUAL INTERVENTION

The API supports:

POST /api/v1/remediation/jobs/{id}/items/{item_id}/manual

for completing a manual item.

So the architecture supports:

Automation
↓
blocked
↓
human
↓
resume

This is called:

Human-in-the-loop automation.

PART XLVII — DURABLE OPT-OUT STATE

After a broker operation, DigiZafe stores broker state.

The Sprint 7 flow includes:

GET /remediation/state

for durable opt-out history.

The configuration includes:

BROKER_OPTOUT_RECHECK_DAYS=90

So the system can avoid blindly repeating an opt-out action if the previous state is still considered fresh.

PART XLVIII — WHY RECHECK?

Suppose:

Day 1
User successfully opts out.

You don't want:

Day 2
Run the same opt-out blindly again.

Instead:

Stored state
↓
Check age
↓
Still fresh?
├── YES → skip unnecessary operation
└── NO → recheck

The documented flow identifies a 90-day fresh-skip period.

PART XLIX — EMAIL CONFIRMATION

The configuration also specifies:

BROKER_EMAIL_CONFIRM_RETRY_DAYS=14

This represents another state-management concept.

Some brokers may require:

Submit request
↓
Receive confirmation email
↓
User/system completes confirmation
↓
Removal process continues

Therefore remediation isn't always a single HTTP/browser event.

It can be a **multi-step stateful process**.

PART L — JOB DEADLINE

The configuration specifies:

BROKER_JOB_DEADLINE_MINUTES=120

This prevents remediation jobs from continuing indefinitely.

Conceptually:

Job starts
↓
deadline timer
↓
work
↓
deadline reached
↓
stop/fail/manual

This is another example of defensive automation.

PART LI — TIMEOUT VS DEADLINE

Professor may ask:

> "What's the difference?"

Runner timeout

Controls one browser operation/run.

Example:

Page load > 90 seconds
→ timeout

Job deadline

Controls the entire remediation job.

Example:

Entire job > 120 minutes
→ deadline exceeded

So:

Timeout = individual execution boundary

Deadline = overall workflow boundary

PART LII — CLOSED-LOOP REMEDIATION

This is one of the strongest architectural concepts.

The workflow doesn't stop after:

Submit opt-out

It continues:

Remediate
↓
Verify
↓
Re-score

The project describes the complete user-facing loop as:

> **Verify → Discover → Explain → Prioritize → Remediate → Re-verify → Re-score**.

This is a powerful point for your presentation.

PART LIII — WHY RE-SCORE?

Suppose before remediation:

Risk = 82

After removing an exposure:

Exposure ↓
Risk ↓

Therefore the platform can update the user's security state.

Conceptually:

Before:
Findings = 10
Risk = 82

Remediation
↓

After:
Findings = 7
Risk = 65

This demonstrates that DigiZafe isn't merely a scraper.

It is a feedback loop.

PART LIV — RE-VERIFICATION

Why not simply trust the success message?

Because:

"Request submitted"

doesn't necessarily mean:

"Data has actually disappeared."

Therefore:

Submit
↓

Broker says request received
↓
Wait / recheck
↓
Verify removal
↓
Update state

The platform exposes:

POST /api/v1/remediation/verify

for broker re-verification.

PART LV — PLAYWRIGHT LIMITATIONS

You absolutely need to know these.

1. DOM changes

Website changes selectors.

Result:

Automation fails.

---

2. CAPTCHA

Automation cannot always proceed.

Result:

Manual intervention.

---

3. Shadow DOM

Standard selectors may fail.

Result:

Manual fallback.

---

4. Cross-origin iframe

Interaction becomes more complicated.

Result:

Possible manual fallback.

---

5. Memory consumption

Chromium is resource-intensive.

Result:

Dedicated worker / limited concurrency.

---

6. External availability

Website could be:

- offline
- slow
- rate-limited
- blocked

Result:

Timeout/retry/failure.

---

7. Heuristic success detection

A textual success hint doesn't mathematically prove deletion.

Result:

Re-verification is still important.

The project's own presentation explicitly lists memory intensity and brittle DOM scraping as major limitations.

PART LVI — IMPORTANT DISTINCTION:

SUBMISSION ≠ REMOVAL

This is an excellent viva point.

Suppose:

POST request successful

It only means:

> "The request was accepted."

It does **not necessarily prove**:

> "The user's data has already been deleted."

Therefore DigiZafe separates:

Request submitted

from:

Removal verified

That is why the platform has re-verification functionality.

PART LVII — SECURITY CONSIDERATIONS

You are not Team Member 4, but your automation engine interacts with sensitive data and external websites.

Therefore understand these principles.

Principle 1 — Minimize data

Only provide the PII required by the broker.

Principle 2 — Isolation

Don't share browser contexts between users.

Principle 3 — Approved targets

Use Green broker registry.

Principle 4 — Worker isolation

Playwright operates in the worker environment.

Principle 5 — Egress controls

External broker requests should pass through the platform's security/egress architecture.

Principle 6 — Auditability

Broker actions should be recorded.

PART LVIII — WHAT IS SSRF?

You may get this question because your worker accesses URLs.

SSRF = **Server-Side Request Forgery**

Imagine an attacker manipulates a URL so that your server requests:

http://127.0.0.1

or an internal service.

Instead of:

Worker
↓
Public broker

you accidentally get:

Worker
↓
Internal network
↓
Sensitive service

That's dangerous.

Team Member 4 owns the security mechanism, but your automation system must operate within it.

PART LIX — WHY NOT ACCEPT ANY URL FROM THE USER?

Because:

User supplied URL
↓
Browser
↓
arbitrary destination

would greatly increase attack surface.

Instead:

Broker ID
↓
Approved registry
↓
Known destination
↓
Automation

is much safer.

PART LX — WHY IS THE PLAYWRIGHT RUNNER NOT IN THE API?

Because the API should remain:

fast
responsive
stateless where possible

while the browser worker can be:

slow
resource-heavy
long-running
failure-prone

Therefore:

FastAPI
↓
Queue
↓
Browser Worker

is a strong separation of concerns.

PART LXI — WHAT DOES THE API DO?

The remediation API supports functionality including:

GET /remediation/brokers
GET /remediation/state
POST /remediation/jobs/broker-optout
GET /remediation/captcha
POST /remediation/captcha/{id}
GET /remediation/freeze
PATCH /remediation/freeze/{id}
POST /remediation/know
POST /remediation/complaints
GET /remediation/requests
POST /remediation/requests/{id}/mark-sent
POST /remediation/verify

The Sprint 7 specification documents these endpoints.

PART LXII — WHAT IS "FREEZE"?

The remediation system isn't limited to data-broker deletion.

It also supports a **freeze checklist**.

Conceptually:

Exposure discovered
↓
Identify relevant protection actions
↓
Freeze checklist
↓
Track completion

The API exposes:

GET /remediation/freeze
PATCH /remediation/freeze/{id}

PART LXIII — RIGHT-TO-KNOW / DELETION REQUESTS

Some situations require generating formal privacy requests rather than automating a browser.

The system supports:

POST /remediation/know

for right-to-know/deletion requests.

It also supports:

POST /remediation/complaints

for complaints.

This is important because:

> **Automation isn't limited to browser clicking.**

The remediation subsystem can also generate structured privacy-rights communications.

PART LXIV — AIDR → DIGIZAFE

Sprint 7 maps concepts from AIDR into DigiZafe.

You don't necessarily need to discuss the source project deeply, but understand the mapping:

AIDR state.json
↓
DigiZafe broker_optout_state

AIDR brokers.js
↓
DigiZafe broker registry + Playwright runner

AIDR verify
↓
DigiZafe /remediation/verify

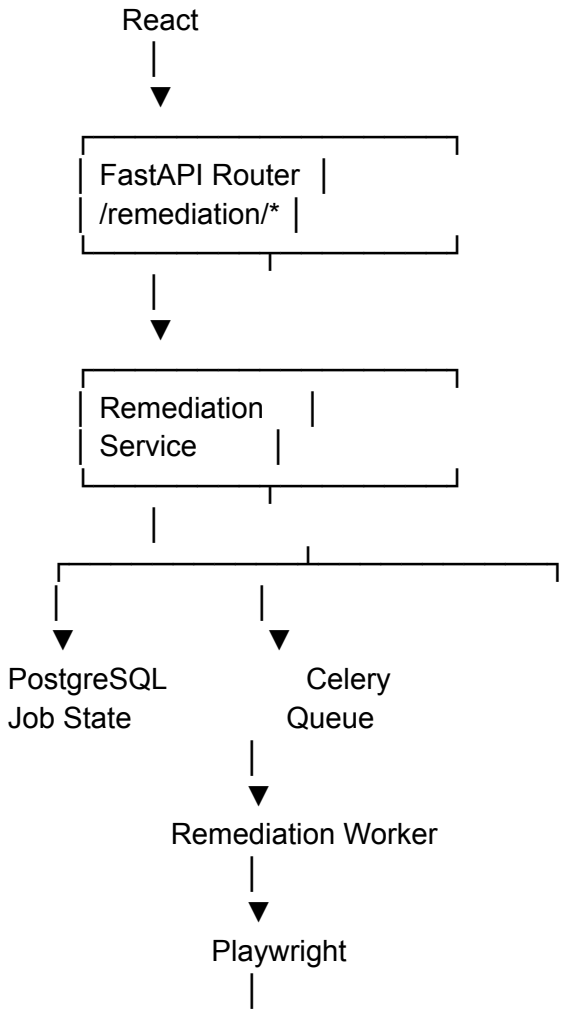
AIDR freeze
↓
DigiZafe /remediation/freeze

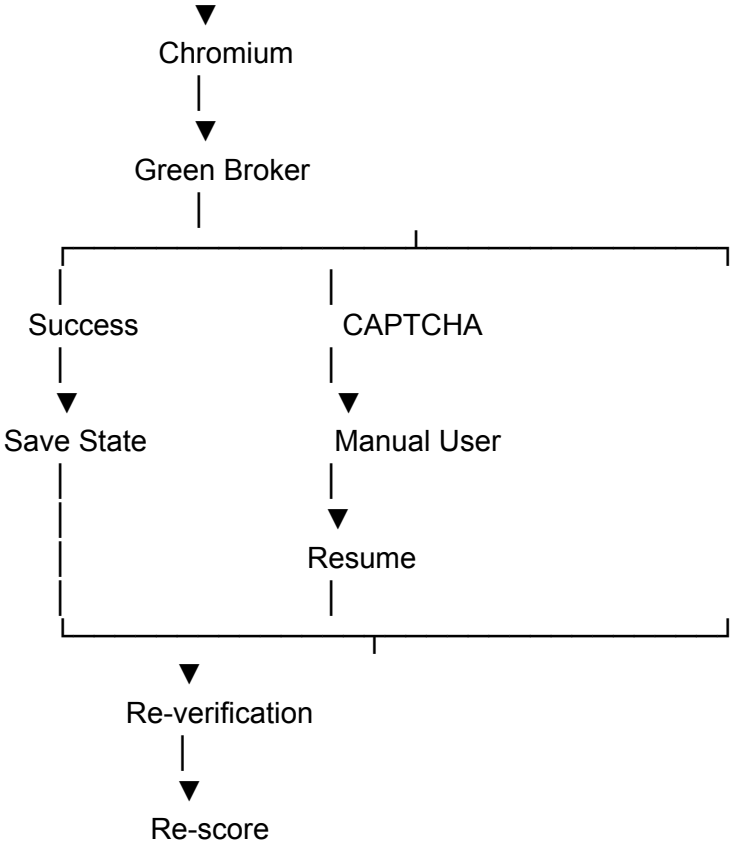
AIDR know
↓
DigiZafe /remediation/know

This mapping is explicitly documented in Sprint 7.

PART LXV — COMPLETE ARCHITECTURE OF YOUR SUBSYSTEM

Memorize this:





PART LXVI — THE MOST IMPORTANT CONCEPT:

HUMAN-IN-THE-LOOP AUTOMATION

Your system is **not trying to automate everything blindly**.

It automates what it can.

When it reaches a boundary:

CAPTCHA
DOM ambiguity
unexpected form

it transitions to:

MANUAL_NEEDED

Then the user completes the required action.

Therefore:

> **The system is best described as human-in-the-loop browser automation rather than fully autonomous browser control.**

This is a much more defensible technical description.

PART LXVII — WHY NOT JUST USE AN API?

Professor:

> "Why do you need Playwright? Why not call an API?"

Answer:

> **"Where a broker exposes a reliable official API, that would generally be preferable. However, many consumer data brokers expose opt-out workflows primarily through web interfaces. Playwright allows DigiZafe to interact with those actual browser-based workflows, including JavaScript-rendered forms. The automation remains restricted to approved broker workflows."**

Excellent answer.

PART LXVIII — WHY NOT USE SELENIUM?

If asked:

> "Why Playwright instead of Selenium?"

A reasonable project-grounded answer:

> **"Playwright provides modern browser automation primitives, including strong locator behavior, automatic waiting and Chromium automation that fits our asynchronous Python worker architecture. For this project, it gave**

> **us a convenient way to automate dynamic JavaScript-heavy broker pages."**

Avoid making unsupported benchmark claims.

PART LXIX — WHY NOT SCRAPY?

Scrapy is excellent for:

large-scale crawling
structured web extraction

But your problem is:

interactive browser workflow

You need:

click
fill
select
JavaScript
browser session
CAPTCHA/manual state
form submission

Therefore Playwright is more appropriate for the remediation layer.

PART LXX — WHY NOT `requests + BeautifulSoup`?

Same reasoning.

Those tools are better suited to:

HTTP retrieval
+
HTML parsing

rather than:

real browser interaction

PART LXXI — YOUR ROLE VS TEAM MEMBER 3

This distinction can save you in a viva.

Team Member 3

> **Finds the data.**

OSINT
↓
Connectors
↓
Findings

You

> **Acts on the data.**

Finding
↓
Remediation
↓
Browser
↓
Opt-out

PART LXXII — YOUR ROLE VS TEAM MEMBER 1

Team Member 1

Determines:

Who does this finding belong to?
↓
How risky is it?

You

Determines:

What action can we take?
↓
Can we submit an opt-out?
↓
Was it successful?

PART LXXIII — YOUR ROLE VS TEAM MEMBER 4

Team Member 4

Protects:

Data
Network
Database
Encryption
Access

You

Builds:

Browser automation

that operates **within those security boundaries**.

PART LXXIV — YOUR ROLE VS TEAM MEMBER 5

Team Member 5

Shows:

Job status
CAPTCHA
Success
Failure
Manual tasks

You

Generate those states in the backend.

Example:

Worker
↓
captcha_needed
↓
Frontend displays CAPTCHA task

PART LXXV — YOUR MOST IMPORTANT FILES

🔴 MUST KNOW

`backend/app/remediation/runners/playwright_runner.py`

Your most important file.

Know:

- browser launch
- context
- page
- navigation

- selectors
- DOM
- CAPTCHA detection
- form filling
- submission
- success hints
- timeout
- fallback

---

backend/app/remediation/broker_registry.py

Know:

- broker loading
- Green broker filtering
- enabled broker filtering
- registry caching

---

backend/app/tasks/remediation_tasks.py

Know:

Celery
↓
worker
↓
Playwright

---

backend/app/services/remediation_service.py

Know:

API
↓
business logic
↓
job creation
↓
worker

🟠 SHOULD KNOW

backend/app/api/v1/remediation.py
backend/app/models/remediation.py
backend/app/schemas/remediation.py
backend/app/repositories/remediation_repository.py
backend/app/domain/remediation_states.py
backend/app/domain/remediation_profile.py

🟡 KNOW CONCEPTUALLY

shared/config/broker_registry/brokers_green.json
templates.py
Dockerfile
docker-compose.yml
Alembic migration
remediation tests

PART LXXVI — 25 QUESTIONS YOU MUST MASTER

Q1. What is Playwright?

> A browser automation framework used to programmatically control Chromium and interact with dynamic web pages.

Q2. Why Playwright?

> Because broker opt-out pages can be JavaScript-heavy and require actual browser interactions rather than simple HTTP requests.

Q3. Why not requests?

Requests retrieves HTTP responses but doesn't provide a complete browser environment for JavaScript-rendered forms and interactive workflows.

Q4. What is DOM?

The Document Object Model, the browser's structured representation of an HTML document.

Q5. What is a locator?

A mechanism used by Playwright to identify an element in the DOM for interaction.

Q6. Why use logical-to-selector mapping?

Because different brokers represent the same logical field with different HTML selectors.

Q7. How do you fill an email?

Identify the configured selector for the broker's email field and use Playwright's input interaction such as `.fill()`.

Q8. Why use `select_option()`?

For HTML `<select>` controls where a value must be selected rather than typed like a normal input.

Q9. How do you detect CAPTCHA?

The runner scans page content for indicators such as `recaptcha`, `hcaptcha`, and `cf-turnstile`.

Q10. Do you bypass CAPTCHA?

No. The current default workflow treats CAPTCHA as a manual/user-assisted intervention point.

Q11. What happens when CAPTCHA is found?

The job transitions to `captcha_needed`, the user receives instructions, completes the CAPTCHA, and the worker resumes.

Q12. How do you know the opt-out succeeded?

The runner examines the resulting DOM/body for configured success hints.

Q13. Is success detection 100% reliable?

No. It is heuristic, which is why re-verification is important.

Q14. What happens if the broker changes its HTML?

The configured selectors may fail and the job should fall back to a manual state rather than pretending the operation succeeded.

Q15. Why are timeouts necessary?

To prevent a slow or unavailable broker from occupying a worker indefinitely.

Q16. Why concurrency = 1?

Chromium is memory-intensive, so limiting concurrency reduces the risk of worker OOM failures.

Q17. Why not run Playwright inside FastAPI?

Browser automation is long-running and resource-intensive, so it belongs in background workers rather than blocking API requests.

Q18. Why isolate browser contexts?

To prevent cookies, local storage, sessions and other browser state from leaking between users.

Q19. What is dry run?

A mode that performs form discovery and filling but does not submit the final opt-out request.

Q20. Why dry run?

To test automation safely without unintentionally executing real deletion/opt-out actions.

Q21. Why Green brokers?

To constrain automated actions to approved workflows rather than allowing arbitrary website automation.

Q22. What happens if the website uses Shadow DOM?

Basic locators may fail; the workflow can fall back to manual intervention.

Q23. What happens after submission?

> The result is interpreted, the remediation state is persisted, and the platform can later re-verify the broker and re-score exposure.

Q24. Does "request submitted" mean data deleted?

> No. Submission acknowledgement and actual removal are different states.

Q25. What is your biggest limitation?

> **Browser automation is inherently brittle and resource-intensive. Changes to broker DOM structures, CAPTCHA mechanisms, embedded forms or website availability can cause automated execution to fail, requiring manual intervention.**

PART LXXVII — 15 HARDER PROFESSOR QUESTIONS

1. How do you prevent infinite waiting?

Use explicit timeouts and job deadlines.

---

2. What happens if the broker is offline?

The browser request times out/fails and the remediation state records the failure rather than hanging indefinitely.

---

3. What if the broker returns HTTP 200 but doesn't remove the data?

HTTP status alone isn't treated as proof. The runner uses success hints and later verification.

---

4. Why not simply use a fixed XPath?

Because broker HTML changes. A configurable selector strategy is more maintainable.

---

5. What if an input is inside an iframe?

Basic page locators may not reach it depending on frame origin/structure; unsupported cases fall back to manual handling.

---

6. Why is browser automation expensive?

Each browser context requires memory and CPU, and JavaScript rendering adds overhead.

---

7. How would you scale it?

Potentially:

Remediation Queue
↓
Multiple Worker Containers
↓
One/few browser contexts per worker

with carefully controlled concurrency and resource limits.

---

8. Why not increase concurrency to 20?

Because browser memory consumption could cause OOM failures. Scaling should be horizontal with resource-aware worker sizing rather than blindly increasing local concurrency.

---

9. What if two users submit the same broker simultaneously?

Each job should use isolated browser contexts and user-scoped persistent state.

---

10. What if a broker requires email confirmation?

The workflow can track confirmation/retry state rather than treating the initial submission as final removal.

---

11. Why keep remediation state in PostgreSQL?

Because remediation state must survive worker restarts and Redis cache eviction.

---

12. What if Redis goes down?

The queueing system is affected, but persistent remediation state should remain in PostgreSQL.

---

13. Why isn't Redis the source of truth?

Redis is used for messaging/cache; durable application state belongs in PostgreSQL.

---

14. How do you prevent malicious PII input from becoming JavaScript?

The backend validates input, and Playwright's `.fill()` treats supplied values as input rather than executable page JavaScript. The project documentation specifically notes Pydantic validation before worker execution.

---

15. How do you test it without deleting real people's information?

Use:

dry_run

test fixtures/mock brokers where applicable, and controlled test environments.

The Sprint 7 verification flow explicitly includes dry-run testing before real jobs.

PART LXXVIII — YOUR 5-MINUTE CONTRIBUTION EXPLANATION

If the professor asks:

> **"Explain your contribution."**

You can say:

> **"My primary responsibility was the automated remediation layer of DigiZafe. The problem we were solving was that discovering personal-data exposure is only half of the problem; users still have to visit data broker websites and manually submit privacy opt-out requests.**
>
> **I built the Playwright-based remediation engine that runs inside asynchronous Celery workers. The API creates a remediation job, the worker receives it through Redis, and the Playwright runner launches an isolated Chromium context for the approved broker.**
>
> **The runner navigates to the broker's opt-out page, waits for the DOM to load, detects CAPTCHA indicators, maps logical fields such as name, email, phone and ZIP to broker-specific selectors, fills the form and submits it when automation is supported. After submission, it examines the resulting DOM for configured success hints.**
>
> **An important design decision was not to assume that every website can be fully automated. If a CAPTCHA, Shadow DOM, iframe or changed website structure prevents reliable automation, the job transitions to a manual state. CAPTCHA therefore becomes a human-in-the-loop step rather than something we blindly try to bypass.**
>
> **I also implemented safety boundaries around the automation. We only automate approved Green brokers, require verified identifiers, isolate browser contexts between users, enforce timeouts and restrict worker concurrency because Chromium is memory-intensive.**
>
> **Finally, remediation is designed as a closed loop: after an opt-out request, the platform persists the state, can re-verify the broker later, and can regenerate the user's risk assessment. So the system doesn't stop at 'request submitted'; it moves toward 'remediation verified and risk re-evaluated.'"**

PART LXXIX — 60-SECOND VERSION

Memorize this if you are short on time:

> **"I worked on DigiZafe's automated remediation engine. I used Playwright to control Chromium and automate privacy opt-out workflows on approved data broker websites. FastAPI creates the remediation job, Redis**

and Celery execute it asynchronously, and the Playwright worker navigates the broker page, identifies the required form fields, fills user-approved PII, handles dynamic DOM behavior and detects CAPTCHA.

The system doesn't assume that every website is automatable. CAPTCHA or unsupported page structures such as complex iframes or changed selectors can move the job into a manual state. We also use isolated browser contexts, hard timeouts, Green broker restrictions and limited worker concurrency to improve security and reliability. After submission, we use success hints for initial result detection and support later broker re-verification and risk re-scoring."

PART LXXX — WHAT YOU MUST MEMORIZE

🔴 ABSOLUTELY MEMORIZE

Playwright
Chromium
DOM
Locator
Celery
Redis
Broker Registry
Green Broker
Remediation Job
captcha_needed
MANUAL_NEEDED
success_hints
dry_run
browser context
timeout
concurrency=1
re-verification

🟠 SHOULD MEMORIZE

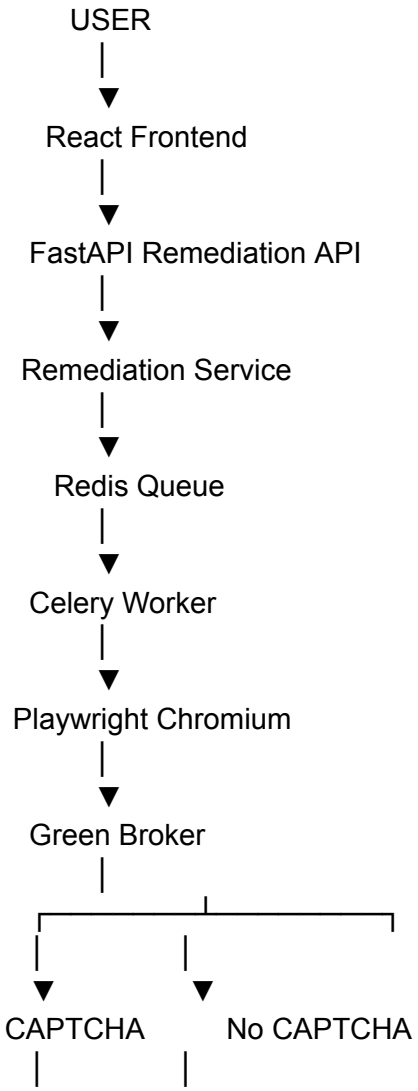
playwright_runner.py
broker_registry.py
remediation_service.py
remediation_tasks.py

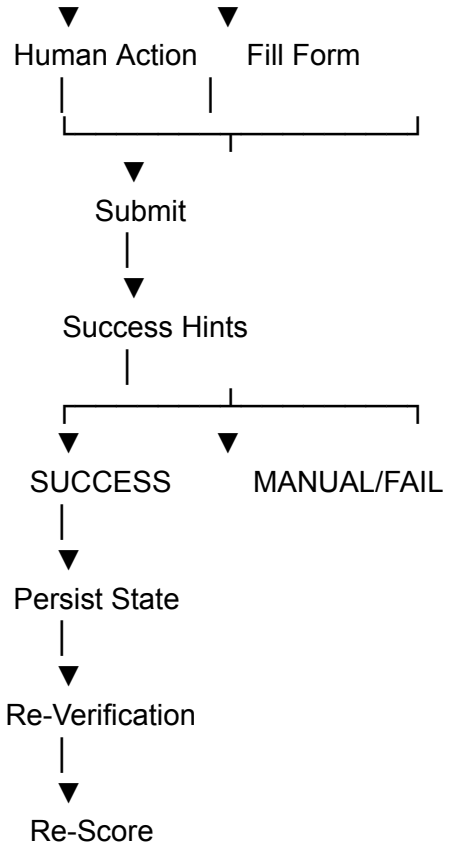
remediation.py
brokers_green.json

CAPTCHA_MODE=manual
FEATURE_CAPSOLVER=false
BROKER_RUNNER_TIMEOUT_SECONDS=90
BROKER_JOB_DEADLINE_MINUTES=120
BROKER_MAX_CONCURRENT_JOBS_PER_USER=1
BROKER_OPTOUT_RECHECK_DAYS=90

The project configuration documents these remediation parameters.

PART LXXXI — THE MOST IMPORTANT DIAGRAM TO REMEMBER





FINAL MENTAL MODEL FOR MEMBER 2

If you remember only **one concept**, remember this:

Your component is not "a web scraper."

It is a:

Human-in-the-loop browser automation and privacy-remediation system.

The distinction is huge.

A scraper says:

> **"I can extract data from websites."**

Your system says:

> **"Given an already-identified exposure and an approved remediation target, I can execute a controlled browser workflow to attempt an opt-out, detect when automation cannot safely continue, involve the user when necessary, persist the result, and eventually verify whether the exposure was actually removed."**

That is a much stronger technical description.

And the complete DigiZafe lifecycle becomes:

DISCOVER
↓
CORRELATE
↓
SCORE
↓
PRIORITIZE
↓
REMEDIATE ← YOUR MAIN COMPONENT
↓
VERIFY
↓
RE-SCORE

The project's documented closed-loop workflow is essentially **Verify → Discover → Explain → Prioritize → Remediate → Re-verify → Re-score**, which is the exact conceptual framework you should keep in your head during the viva.